

LINFO1131 August 2022 Exam

Guillaume Gillet

TOTAL POINTS

8 / 30

QUESTION 1

Power of lazy evaluation 10 pts

1.1 Definition of lazy quicksort 0 / 2

- 0 pts Correct
- 0.25 pts Recursion problem in partition
- 0.5 pts Problem in lquicksort
- 0.25 pts Base case problem in partition
- ✓ - 2 pts Incorrect
 - 0.5 pts Partition missing
 - 0.5 pts LAppend missing

1.2 Translation of lazy quicksort 1.5 / 3

- 0 pts Correct
- 0.5 pts Error in lquicksort
- 0.5 pts Error in lappend
- 1.5 pts No threads or incorrect threads
- 1.5 pts No WaitNeeded
- ✓ - 1.5 pts No bodies or incorrect bodies
 - 3 pts Incorrect
 - 0.5 pts Arguments incorrect

1.3 Execution of lazy quicksort 0 / 5

- 0 pts Correct
- 2 pts Missing information on some executed procedures
- ✓ - 5 pts Incorrect
 - 3 pts Explanation too sketchy on executed procedures
 - 1 pts Missing explanation for LAppend activation
 - 4 pts One step ok, rest is missing

QUESTION 2

2 Flavius Josephus problem 0 / 5

- 0 pts Correct

✓ - 5 pts Incorrect

- 2 pts Protocol incorrect or missing
- 2 pts Deterministic dataflow program missing
- 1 pts No explanation of short-circuit for the program

- 1 pts Partial explanation of protocol

☞ You give an implementation of monitors, this has nothing to do with the Flavius Josephus problem.

QUESTION 3

3 Reentrant locks 3.5 / 5

- 0 pts Correct
- ✓ - 1 pts Specification is incomplete
 - 2 pts No implementation
- ✓ - 0.5 pts Explanation of need for reentrancy is incomplete
 - 0.5 pts Small errors in implementation

QUESTION 4

4 Deadlocks 1 / 5

- 0 pts Correct
- 0.5 pts Definition of deadlock is incomplete
- 0.5 pts Error in definition of wait-for graph
- ✓ - 1 pts Incorrect or missing definition of deadlock
- ✓ - 2 pts Incorrect or missing definition of wait-for graph
- ✓ - 1 pts Error in example
 - 0.5 pts Cycle cannot be infinite

QUESTION 5

5 Serializability (isolation) 2 / 5

- 0 pts Correct
- 1 pts Incorrect definition of serializability
- ✓ - 2 pts No scenario for naive transaction manager

- ✓ - **1 pts** Serializability scenario is incomplete
- **0.5 pts** Incomplete definition of serializability
- **2 pts** No scenario for serializability

Final exam LINFO1131

August 24, 2022
Prof. Peter Van Roy

First and last name	GILLET GUILLAUME
NOMA	26361900

Q1: The power of lazy evaluation [10pts] (Q1 corresponds to the midterm)

Q1.1: Definition of lazy quicksort [2pts]. The lazy version of quicksort allows to compute the k smallest elements out of n elements with a very good upper bound on time complexity. Give the Oz code for lazy quicksort, which consists of three routines Partition, LAppend, and LQuicksort, where LAppend and LQuicksort are lazy and Partition is not. Assume the procedure {Partition L X L1 L2} partitions L into L1 (elements less than X) and L2 (elements greater or equal to X).

```

Lazy {quicksort K N}
  if K == 0 then nil
  else
    S = {Lquicksort N}
    cons S of
      [H | T] then H | {quicksort K-1 T}
    end
  end
end

```

Q1.2: Translation of lazy quicksort [3pts]. Translate LAppend and LQuicksort into procedures that use WaitNeeded together with a thread. [1pt]

```

proc {Lappend L1 L2 ?R}
  thread {WaitNeeded R} Lappend
end

proc {Lquicksort L ?R}
  thread {WaitNeeded R} Lquicksort
end

```

First and last name *Gillet Guillaume*

NOMA *26361900*

Q1.3: Execution of lazy quicksort [5pts].

Show the full execution of the statements:

$S = \{\text{LQuicksort } [7\ 3\ 2\ 8\ 6\ 4]\}$

$\{\text{Browse } S.1\}$

Show exactly what procedures are executed when S.1 is done. The operation S.1 needs S and will therefore activate a blocked $\{\text{WaitNeeded } S\}$ call. (Similarly for **case S of HIT**, which needs S as well.) With this semantics, show exactly what procedures are executed until the S.1 operation completes and gives a value to Browse. What value is displayed by Browse?

Browse will display [3264]

First and last name *Gillt Guillaum*

NOMA *26361800*

Q2: Flavius Josephus problem [5pts]

For this question, answer the following three items:

1. Roman historian Flavius Josephus explains that he was in a cave with N soldiers put in a circle where every K th soldier counted was supposed to commit suicide, and he was the last survivor. Do you believe his story? For this item, you are given a ring of N agents and you should specify a multi-agent protocol that implements the Flavius story.
2. Write a deterministic dataflow program that implements this protocol. Each agent is a stream object with one input and one output stream, and N agents should be created and connected in a ring.
3. Your program should terminate each agent and remove it from the ring when the agent commits suicide. Explain how your program does this.

1). Wait: Restart thread $\rightarrow$ release lock $\rightarrow$ put it in the wait set $\rightarrow$ wait returns and try to get a lock.

Notify: on Agent try to get the lock.

Notify All: All the agent try to get the lock.

$\hookrightarrow$ if it's the special agent get the lock

$\hookrightarrow$ if it's not put it in the wait set.

```

2)
gen {New queue.}
  L = {New lock}
  xc = {New cell q(0 x x)}
  Proc {insert x}
  end
  Proc {delete x}
  end
  in q(insert: insert, delete: delete)
end
end

```

```

Proc {insert x}
  NF B2 in
  lock L then F
  @C = q(N F x | B2)
  C := q(N+1 F B2)
end
end

```

```

Proc {delete x}
  NF B2 in
  lock L then F
  @C := q(N F | B2)
  C := q(N-1 F B2)
end
end

```

Create a queue where we can insert and delete (suicide) each agent

3) queue on agent se suicide (get delete) on notify All.
 $\hookrightarrow$ get delete one deleted

First and last name *Gilbert Guillaume*

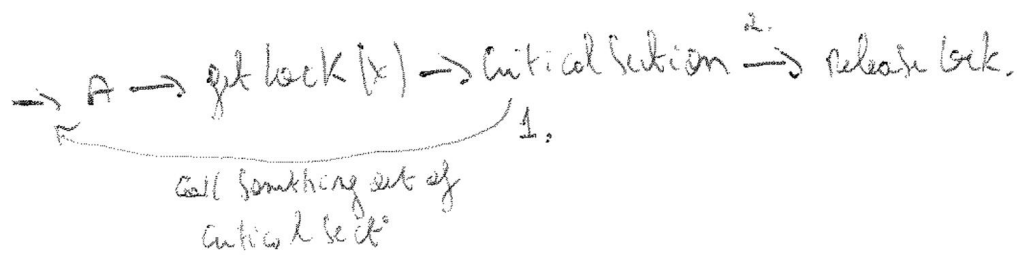
NOMA *16361800*

Q3: Reentrant locks [5pts]

For this question, answer the following three items:

1. Specify a reentrant lock.
2. Explain why reentrancy is important. Give a scenario where it is needed.
3. Give an implementation of the reentrant lock, similar to what we saw in the course.

1) A reentrant lock is a lock where the thread who has this lock can call the ^{in a 3rd} ~~unit~~ (use the lock) again.
for example: a thread *A* ^{in a 3rd} get a lock *x*, enter a critical section and this section. call the ^{in a 3rd} ~~unit~~ again, *A* is not going to be blocked and can reenter the critical section:



2) Reentrancy is very important to avoid dead locks. (look example above), it's also needed in producer/consumer problems

```
3) fun {Newlock}
  Token = {new cell Unit}
  curthr = {new cell Unit}
  {lock P}
  :
in
  lock' (lock' lock) end
end

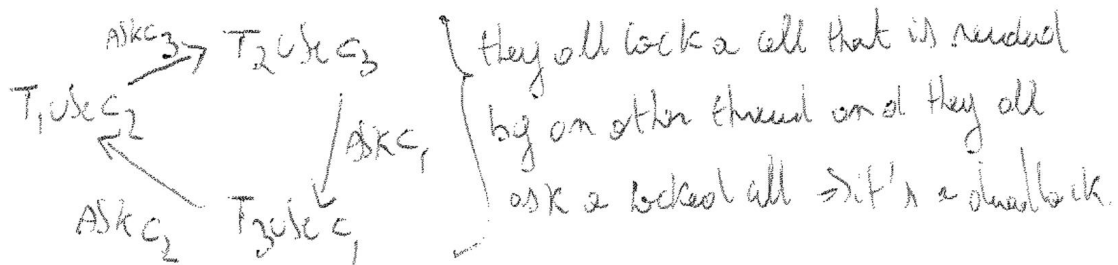
proc {lock P}
  if curthr = {Thread.Thin} then {P}
  else old new in
    {exchange Token old new}
    {wait old}
    curthr = {Thread.Thin}
  end
  try {P} finally
    curthr = Unit
    new = Unit
  end
end
```

First and last name Gilbert Guillaume

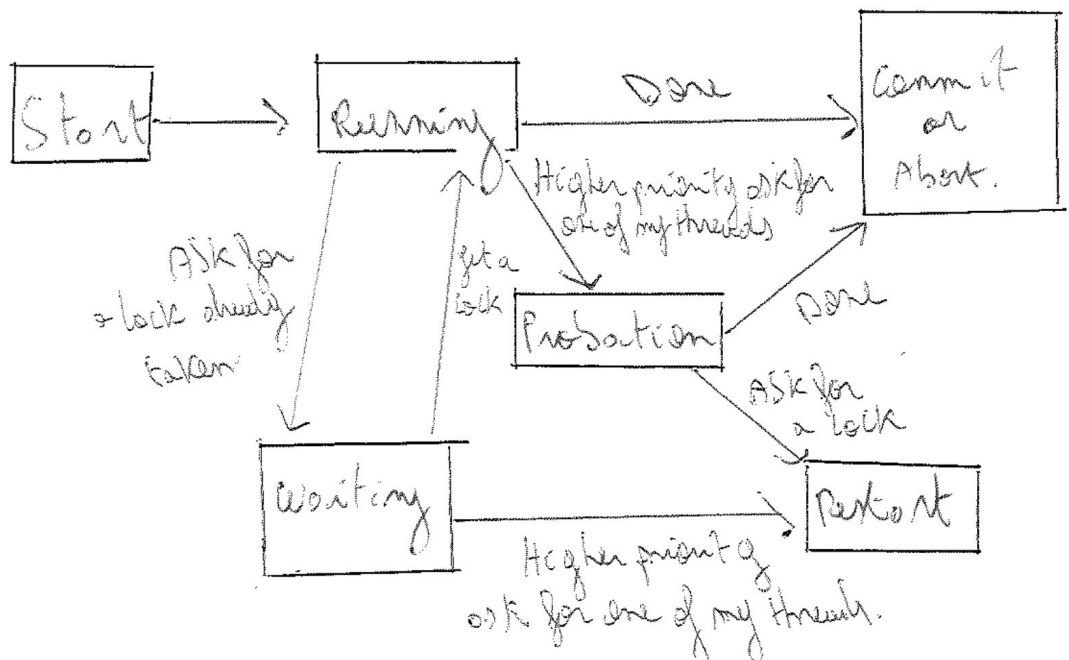
NOMA 2636 1900

Q4: Deadlocks [5pts]

Give a formal definition of deadlock using the wait-for graph. As part of your answer, define the wait-for graph. Give an example of a deadlock with three transactions, T1, T2, and T3, where T1 uses cells C2 and C3, T2 uses C3 and C1, and T3 uses C1 and C2.



- deadlock is when a situation is blocked because threads ask for a lock already taken that will never be released.
- to avoid this situation, we can have threads / transactions with priority (schema under)



First and last name G. Luk G. WILLAUM

NOMA 16361800

Q5. Serializability (isolation) [5pts]

Answer the following three items:

1. Define the serializability property of transactions.
2. Assume a naïve transaction manager that immediately gives an unused lock to any transaction that requests it and releases the lock when the transaction no longer needs it. Give a scenario where serializability is not satisfied.
3. Explain the technique of two-phase locking. Reformulate the scenario of the previous item using two-phase locking and show that serializability is satisfied.

1) Transaction should not interfere with other transactions.
(concurrent)

↳ Comes from ACID definition

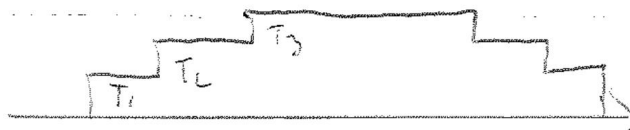
↳ A update in 1 opinst^o (Atomic)

C Repeat invariant (consistent)

I 2 concurrent transact^o must not interfere with each other (Isolated)

D Survive to crash. (Durability)

2) in a consumer/producer problems



3) Two phase locking, we give and release all the locks at the same time.